

Question 1.

What is a Functional Interface? What annotation is used to mark a Functional Interface, and is this annotation mandatory? Give two examples of built-in Functional Interfaces in Java 8 and explain their abstract method signatures.

A functional interface has exactly one abstract method and can be implemented using a lambda expression. The `@FunctionalInterface` annotation is optional.

Two Java 8 examples are:

`Predicate<T>`: `boolean test(T t)` takes a value of type `T` and returns a boolean.

`Function<T, R>`: `R apply(T t)` takes a value of type `T` and returns a result of type `R`.

Question 2.

Hello from Greeting

And hello from Person

Person overrides `sayHello()` to resolve the conflict between the two interfaces. It first calls `Greeting.super.sayHello()`, which prints the first line, then prints its own message. The method in `Farewell` is not called.

Question 3.

```
Comparator<String> comparator =  
    (s1, s2) -> s1.length() - s2.length();
```

Remove the anonymous class declaration, annotation, and method declaration. Keep the parameters and add `->`. Parameter types are inferred. Since the body returns a single expression, omit the braces and return.

Question 4.

A, D, E, and F are valid.

B is invalid because return requires a block enclosed in braces.

C is invalid because its block must return a value, and `x * 2;` is not a valid standalone statement.

Question 5.

1-B: Bound instance: Calls `println` on the specific object `System.out`.

2-D: Unbound instance: The input string is the object on which `toUpperCase()` is called.

3-C: Static: References the static method `Math.abs`.

4-A: Constructor: Creates a new `ArrayList`.

5-E: Unbound instance: The first argument is the object on which `compareTo` is called, and the second is its method argument.

Question 6.

`Optional.of()` throws a `NullPointerException` if the value is null, while `Optional.ofNullable()` returns an empty `Optional`. In this code, `Optional.of(value)` throws an exception, so neither print statement is executed. If evaluated separately, `opt2.isPresent()` would return false.

Question 7.

--- Using `orElse` ---

Creating default value

Result: Hello

--- Using `orElseGet` ---

Result: Hello

`orElse()` evaluates `createDefault()` even though the `Optional` contains "Hello", so it prints "Creating default value". `orElseGet()` calls its supplier only when the `Optional` is empty, so `createDefault()` is not called. Both return "Hello".

Question 8.

```
List<String> allProductNames = orders.stream()
    .flatMap(order -> order.getProducts().stream())
    .map(Product::getName)
    .collect(Collectors.toList());
```

`map()` transforms each element into one result. `flatMap()` transforms each element into a stream and flattens those streams into a single stream. Here, `flatMap()` combines the products from all orders, `map()` extracts their names, and `collect()` gathers them into a list.

Question 9.

```
Stream created
Calling findFirst...
Filtering: 1
Filtering: 2
Mapping: 2
Result: 20
```

Stream operations are lazy: `filter()` and `map()` run only when `findFirst()` is called. 1 is rejected, and 2 is mapped to 20. Processing then stops because the first result has been found.

Question 10.

Using `Optional` as a parameter is unnecessary here; accept a list and use an empty list when there are no products.

A method returning `Optional` should return `Optional.empty()`, never `null`.

`isPresent()` followed by `get()` is safe here, but can be simplified.

```
public void processProducts(List<Product> products) {  
    products.forEach(this::process);  
}  
  
public Optional<BigDecimal> calculateTotal(List<Product> products) {  
    if (products == null || products.isEmpty()) {  
        return Optional.empty();  
    }  
  
    BigDecimal total = products.stream()  
        .map(Product::getPrice)  
        .reduce(BigDecimal.ZERO, BigDecimal::add);  
  
    return Optional.of(total);  
}
```